PROYECTO FINAL · PRÁCTICAS PROFESIONALIZANTES I

Simulador de cajero automático

Para que las personas mayores practiquen sin riesgo

1 · Qué es y qué hace

2 · Cómo está hecho

3 · Calidad y próximos pasos

TEMA 1

Practicar sin miedo a equivocarse

Para quién

Personas mayores que quieren aprender a usar un cajero sin miedo a equivocarse.

Qué es

Un simulador web que imita un cajero real: pantalla, botones, teclado, tarjeta, billetes y comprobante. No se usa dinero real.

Cómo está hecho

HTML, CSS y JavaScript puros. Sin librerías ni servidor: se abre con doble clic en index.html.

TEMA 1

Qué puede hacer el usuario

Ingresar	Inserta la tarjeta y escribe su clave de 4 dígitos (3 intentos)
Consultar	Ve el saldo y los últimos movimientos
Extraer efectivo	Monto fijo u otro monto; los billetes salen en la ranura
Transferir	Por CBU o por alias, con historial
Pagar y recargar	Servicios, celular y SUBE
Cambiar la clave	Con bloqueo de la tarjeta tras 3 errores
Comprobante	El ticket sale en la impresora del cajero

TEMA 2

Cuatro capas, cada una con su trabajo

1 · Interfaz

index.html y CSS: todas las pantallas escritas, solo una visible a la vez.

2 · Eventos

ui.js recibe clics y envíos y ejecuta la función que corresponde.

3 · Pantallas

Leen lo que escribió el usuario, piden el trabajo al cajero y muestran el resultado.

4 · Lógica

Clase CajeroAutomatico con las reglas y los datos. No toca el HTML.

La interfaz nunca llama directo a la lógica: todo pasa por las pantallas.

TEMA 2

Un listener conecta todos los botones

ATRIBUTO EN EL HTML	QUÉ PASA	EJEMPLO
`data-destino`	Muestra otra pantalla	`mostrarPantalla("pantallaPagos")`
`data-accion`	Ejecuta una función registrada	`accionesBoton.consultarSaldo`
`data-operacion`	Procesa un formulario	`manejadoresFormulario.transferirCbu`

El cajero responde true o false y deja el motivo en motivoRechazo. La pantalla decide qué mostrar.

TEMA 3

Cómo lo probamos y cómo trabajamos

Pruebas

41 pruebas automáticas sobre la lógica, con Node, y 22 casos manuales para la interfaz.

Decisiones

Un solo HTML con pantallas ocultas, lógica separada de la pantalla y estado en memoria.

Equipo

Repositorio Git con las convenciones del README, requerimientos en planilla y tareas repartidas por archivo.

TEMA 3 · CIERRE

Qué sigue y consultas

Próximos pasos

- Asistente guiado en texto y voz
- Sonidos del cajero
- Despliegue en GitHub Pages
- Más servicios y recargas

Preguntas

¿Por qué un solo HTML?
Mantiene el estado y evita repetir el marco del cajero.

¿Dónde está la lógica?
En la clase CajeroAutomatico, que no toca el HTML.

¿Y si cancelo una extracción?
No se descuenta nada: se ejecuta al terminar la espera.